

THE ULTIMATE DEVOPS & SRE COMPENDIUM

Complete DevOps Master Handbook

From Beginner to Principal Engineer

Exhaustive Command Reference • Production Architecture • Runnable Automation Scripts • Cloud & SRE Blueprints

Curriculum Coverage	12 Comprehensive Domains: Linux Administration & Kernel • Networking & Security Hardening • Advanced Git & GitOps • Defensive Bash Automation • Python SRE Scripting • Docker & Container Internals • Kubernetes Orchestration & Helm • Terraform & OpenTofu IaC • Ansible Configuration Management • CI/CD Pipelines (GitHub Actions & Jenkins) • Observability (Prometheus, PromQL, Grafana, Logs) • Enterprise Capstone & Production Readiness.
Structure	Contains every essential command, flag, parameter, configuration file, and production manifest without omissions. Includes full production scripts for backup, auto-healing, multi-stage containers, Kubernetes deployments, Terraform VPCs, Ansible roles, and CI/CD pipelines.
Standard	Modern Tooling Standard: Linux Kernel 6+, Kubernetes v1.30+, Docker Compose v2, Terraform v1.8+, GitHub Actions, Prometheus v2.50+.

Handbook Contents & Index:

- Module 1: Linux SysAdmin, Kernel & Storage Internals
- Module 2: Networking, Firewalls & Security Hardening
- Module 3: Version Control & Advanced Git Workflows
- Module 4: Defensive Shell Scripting (Bash) Automation
- Module 5: Python for DevOps, Cloud & SRE Automation
- Module 6: Containerization with Docker & Compose Stacks
- Module 7: Kubernetes Orchestration & Helm Packaging
- Module 8: Infrastructure as Code (IaC) with Terraform
- Module 9: Configuration Management with Ansible
- Module 10: Enterprise CI/CD Pipelines (GHA & Jenkins)
- Module 11: Observability, PromQL & Production Debugging
- Module 12: Enterprise Capstone & 12-Week Roadmap

Module 1: Linux SysAdmin, Kernel & Storage Internals

Linux is the foundation of all cloud computing. This module details every core command for file systems, ownership, ACLs, stream manipulation, systemd services, and hardware resource monitoring.

1.1 File System Navigation, Hierarchy & Disk Operations

<code>pwd</code>	Print absolute path of current working directory.
<code>ls -lha --time-style=long-iso</code>	Detailed file list including hidden items, ISO timestamps, permissions, and sizes.
<code>cd - / cd ~ / cd /var/log</code>	Switch to previous directory / user home / system log root.
<code>mkdir -p /opt/app/{bin,config,logs,data}</code>	Create nested directory trees in a single atomic invocation.
<code>cp -av /src /dest_backup</code>	Archive copy preserving file attributes, timestamps, ownership, and symlinks.
<code>mv -u source.tar /storage/</code>	Move file only when source is newer than destination or missing.
<code>rm -rf /tmp/cache_*</code>	Recursively and forcefully delete directories and matching files.
<code>ln -s /var/log/app/app.log /home/ubuntu/app.log</code>	Create symbolic link for easy log monitoring without moving files.
<code>ln /var/data/source.db /var/backup/hardlink.db</code>	Create hardlink pointing directly to identical inode on same filesystem.
<code>find /var/log -type f -name '*.log' -mtime +14 -size +50M</code>	Find <code>`.log`</code> files older than 14 days and larger than 50MB.
<code>find /opt/app -type f -exec chmod 644 {} +</code>	Find all regular files and set permissions to 644 efficiently.
<code>find /opt/app -type d -exec chmod 755 {} +</code>	Find all directories and set permissions to 755.
<code>df -hT --exclude-type=tmpfs</code>	Display total, used, and available disk space per filesystem with types.
<code>du -ah --max-depth=1 /var sort -hr head -n 15</code>	Inspect largest directory space consumers sorted descending.
<code>lsblk -fm</code>	List block devices with filesystem UUIDs, mount points, and permissions.
<code>fdisk -l /dev/nvme0n1</code>	View disk partition layout and sector geometries on NVMe storage.

1.2 File Permissions, Advanced ACLs & User Management

<code>chmod 755 script.sh</code>	Set rwxr-xr-x (Owner: read/write/execute; Group & Others: read/execute).
<code>chmod 600 ~/.ssh/id_rsa</code>	Restrict SSH private key permissions strictly to owner read/write.
<code>chmod u+s /usr/local/bin/custom_tool</code>	Set SUID bit: binary executes with permissions of the file owner.
<code>chmod g+s /var/shared_dir</code>	Set SGID bit: new files inside directory inherit group ownership automatically.
<code>chmod 1777 /tmp</code>	Set Sticky bit: users can only delete files they personally own.
<code>chown -R www-data:www-data /var/www/html</code>	Recursively set user and group ownership to www-data.
<code>setfacl -m u:deployer:rwx /opt/data</code>	Assign Access Control List (ACL) granting user 'deployer' custom read/write/exec.
<code>getfacl /opt/data</code>	Display complete POSIX Access Control Lists for file or directory.
<code>useradd -m -s /bin/bash -G sudo,docker devops</code>	Create user 'devops' with home directory, bash shell, and sudo/docker groups.
<code>usermod -aG sudo ubuntu</code>	Append existing user to sudo superuser administrative group.
<code>userdel -r tempuser</code>	Delete user account and completely purge associated home directory.
<code>passwd -l devops</code> / <code>passwd -u devops</code>	Lock or unlock user password authentication.
<code>sudo visudo -c</code>	Check syntax and safely edit sudoers configuration file (/etc/sudoers).

1.3 Text Processing, Regex & Stream Manipulation

<code>grep -rE 'ERROR FATAL EXCEPTION' /var/log/app/</code>	Search recursively for regex patterns with matching line numbers.
<code>grep -v '^#' /etc/nginx/nginx.conf</code> <code>grep -v '^\$'</code>	Filter out all comment lines and empty blank lines.
<code>awk '\$9 >= 500 {print \$1, \$7, \$9}' access.log</code>	Parse log columns: print client IP, requested URL, and HTTP 5xx error status.
<code>awk -F: '{print \$1, \$3}' /etc/passwd</code>	Extract usernames and UIDs using colon ':' as field delimiter.
<code>sed -i 's/DEBUG/INFO/g' /opt/app/config.env</code>	Perform in-place global string substitution across file.
<code>sed -n '50,100p' /var/log/syslog</code>	Extract and print specific line range (lines 50 to 100).
<code>cut -d ' ' -f 1 access.log</code> <code>sort</code> <code>uniq -c</code> <code>sort -nr</code> <code>head -n 10</code>	Extract top 10 highest-traffic client IP addresses hitting the server.
<code>tr '[:lower:]' '[:upper:]' < input.txt > output.txt</code>	Translate all lowercase text to uppercase.
<code>paste -d, file1.txt file2.txt</code>	Merge lines of two files horizontally with comma delimiter.
<code>tee -a /var/log/app.log</code>	Redirect standard output to terminal display and append to file simultaneously.
<code>xargs -I {} -P 4 gzip {}</code>	Execute parallel compression tasks across 4 CPU cores using xargs.

1.4 Process Management, Systemd & Performance Diagnostics

<code>ps aux grep [n]ode</code>	List all active node processes without matching the grep process itself.
<code>ps -ef --forest</code>	Display complete hierarchical process tree showing parent-child relationships.
<code>top -b -n 1 head -n 25</code>	Batch mode single snapshot of system load and top processes.
<code>htop / btop</code>	Interactive real-time visual system resource and process manager.
<code>kill -15</code>	Send SIGTERM (graceful termination signal) allowing process to clean up.
<code>kill -9</code>	Send SIGKILL (immediate kernel-enforced process termination).
<code>pkill -u deployer -9</code>	Force kill all processes owned by user 'deployer'.
<code>nohup ./server > server.log 2>&1 &</code>	Run process immune to hangups in background redirecting stdout & stderr.
<code>systemctl list-unit-files --type=service --state=enabled</code>	List all services configured to start automatically on system boot.
<code>systemctl daemon-reload</code>	Reload systemd manager configuration after editing unit files.
<code>systemctl enable --now nginx</code>	Enable nginx at boot and start service immediately.
<code>systemctl restart reload stop nginx</code>	Restart, hot-reload configuration without dropping connections, or stop service.
<code>journalctl -u nginx.service -f --since '1 hour ago'</code>	Follow live systemd service logs for nginx starting 1 hour ago.
<code>journalctl -p err -b</code>	View all error-priority log messages from current system boot.
<code>free -h</code>	Display total, used, free, cached, and available RAM and swap memory.
<code>vmstat 1 5</code>	Display virtual memory, paging, CPU context switches, and interrupts.
<code>iostat -xz 1 3</code>	Display extended disk I/O utilization, queue length, and service time.
<code>dmesg -T grep -E 'oom-killer Out of memory'</code>	Inspect kernel ring buffer for Out-Of-Memory killed processes.
<code>uptime</code>	Display current system time, uptime duration, active users, and 1/5/15m load averages.

Module 2: Networking, Firewalls & Security Hardening

Production reliability and security mandate rigorous network diagnostics, port socket filtering, firewall rule enforcement, and hardened SSH encryption.

2.1 Network Diagnostics & Socket Inspection

<code>ip addr show / ip route show</code>	Inspect IP addresses on interfaces and kernel IP routing tables.
<code>ip link set eth0 up/down</code>	Bring network interface administratively up or down.
<code>ss -tulnp</code>	List all listening TCP/UDP sockets, bound local ports, and associated process PIDs.
<code>ss -s</code>	Display total socket statistics summary (ESTABLISHED, CLOSED, TIME-WAIT).
<code>netstat -rn</code>	Display routing table in numerical format.
<code>ping -c 4 -i 0.2 8.8.8.8</code>	Send 4 ICMP echo requests with 200ms interval to test latency.
<code>tracert -n -m 30 api.company.com</code>	Trace network route hops to target host with 30 max hops without reverse DNS.
<code>mtr --report -c 10 api.company.com</code>	Combined ping and traceroute diagnostic report over 10 cycles.
<code>dig +short +trace api.company.com</code>	Perform hierarchical recursive DNS resolution trace from root servers to leaf.
<code>dig -x 1.1.1.1 +short</code>	Reverse DNS lookup to resolve PTR record for IP address.
<code>nslookup -type=MX company.com</code>	Query mail exchange (MX) DNS records for domain.
<code>curl -Iv https://api.company.com</code>	Inspect HTTP/HTTPS request, response headers, status codes, and SSL cert chain.
<code>curl -s -o /dev/null -w '%{http_code} %{time_total}s\n' https://api.company.com</code>	Measure exact HTTP response status code and total roundtrip response latency.
<code>curl -X POST -H 'Content-Type: application/json' -d '{"status":"ok"}' https://api.com/v1</code>	Perform HTTP POST request with JSON payload.
<code>nc -zv -w 3 10.0.0.15 5432</code>	Test TCP connection to database port with 3-second timeout.
<code>tcpdump -i eth0 -nn -s0 -w capture.pcap 'port 80 or port 443'</code>	Capture full payload network packets on interface for Wireshark analysis.

2.2 Firewall Rules & Traffic Control (UFW & iptables)

<code>sudo ufw default deny incoming</code>	Set baseline policy to deny all unsolicited incoming connections.
<code>sudo ufw default allow outgoing</code>	Allow all outgoing traffic initiated from the server.
<code>sudo ufw allow 22/tcp comment 'SSH'</code>	Open port 22 for SSH with administrative comment.
<code>sudo ufw allow from 10.0.0.0/16 to any port 5432 proto tcp</code>	Allow PostgreSQL access strictly from internal VPC CIDR block.
<code>sudo ufw delete allow 8080/tcp</code>	Delete specific firewall rule.
<code>sudo ufw enable && sudo ufw status verbose</code>	Enable firewall and display all active rules with numbered indices.
<code>sudo iptables -L -v -n</code>	List all active kernel iptables filter rules with packet counters.
<code>sudo iptables -A INPUT -p tcp --dport 80 -j ACCEPT</code>	Append iptables rule allowing incoming HTTP traffic.
<code>sudo iptables -A INPUT -s 192.168.1.100 -j DROP</code>	Block all traffic originating from specific rogue IP.

2.3 SSH Administration, Hardening & Secure Tunneling

<code>ssh-keygen -t ed25519 -C 'devops_prod_key'</code>	Generate modern, high-security Ed25519 elliptic curve keypair.
<code>ssh-copy-id -i ~/.ssh/id_ed25519.pub user@server_ip</code>	Safely append public key to remote host ~/.ssh/authorized_keys.
<code>ssh -i ~/.ssh/id_ed25519 -p 2222 user@server_ip</code>	Connect via SSH specifying custom private key identity and custom port.
<code>ssh -L 9000:localhost:9000 -N -f user@remote_server</code>	Local port forward: access remote server port 9000 locally via encrypted tunnel.
<code>ssh -R 8080:localhost:3000 user@public_bastion</code>	Remote reverse port forward: expose local dev server 3000 on public bastion 8080.
<code>ssh -D 1080 -N -f user@bastion</code>	Create dynamic SOCKS5 proxy tunnel on local port 1080 routing through bastion.
<code>rsync -avzP --exclude='.git' ./build/ user@server:/var/www/</code>	Fast, compressed, differential directory synchronization over SSH.
<code>scp -P 2222 -r ./dist user@server:/opt/app/</code>	Securely copy directories over SSH using custom port.

Production Hardened SSH Daemon Config (/etc/ssh/sshd_config):

```
Port 2222
Protocol 2
PermitRootLogin no
PasswordAuthentication no
PubkeyAuthentication yes
AuthorizedKeysFile .ssh/authorized_keys
ChallengeResponseAuthentication no
UsePAM yes

X11Forwarding no
MaxAuthTries 3
ClientAliveInterval 300
ClientAliveCountMax 2
AllowGroups sudo devops
```

Module 3: Version Control & Advanced Git Workflows

Git provides immutable version control and forms the foundation for GitOps, continuous delivery, and infrastructure collaboration.

3.1 Repository Initialization, Staging & Commit Management

<code>git config --global user.name 'DevOps Engineer'</code>	Set global author name for commits.
<code>git config --global user.email 'devops@company.com'</code>	Set global author email address for commits.
<code>git init -b main</code>	Initialize new git repository with default branch named 'main'.
<code>git clone --depth 1 --branch main</code>	Shallow clone fetching only the latest commit to optimize CI pipeline speed.
<code>git status -sb</code>	Display compact, short-format status with branch tracking info.
<code>git add . / git add -A</code>	Stage modified and newly created files across entire repository.
<code>git add -p</code>	Interactively review and stage individual change hunks.
<code>git commit -m 'feat(auth): implement oauth2 token validation'</code>	Create commit following Conventional Commits format.
<code>git commit --amend --no-edit</code>	Append staged changes into previous commit without altering commit message.
<code>git log --oneline --graph --decorate --all</code>	Visual ASCII representation of all branches and commit histories.
<code>git diff / git diff --staged</code>	Inspect unstaged vs staged line-by-line modifications.
<code>git diff main..feature/payments</code>	Compare entire diff between main branch and feature branch.
<code>git show</code>	Display full metadata and patch diff for specific commit.

3.2 Branching, Merging, Rebasing & Remote Synchronization

<code>git branch -a</code>	List all local and tracking remote branches.
<code>git switch -c feature/orders / git checkout -b feature/orders</code>	Create and switch to a new feature branch in one step.
<code>git switch main / git checkout main</code>	Switch back to main branch.
<code>git branch -d feature/orders</code>	Safely delete merged feature branch locally.
<code>git branch -D feature/orders</code>	Force delete unmerged feature branch locally.
<code>git push origin --delete feature/orders</code>	Delete branch on remote repository.
<code>git fetch --all --prune</code>	Fetch all remote refs and delete local references to deleted remote branches.
<code>git pull --rebase origin main</code>	Pull upstream commits and replay local unpushed commits on top.
<code>git merge --no-ff feature/orders</code>	Merge feature branch with explicit merge commit preserving history.
<code>git rebase main</code>	Replay current branch commits cleanly on top of latest main branch.
<code>git rebase -i HEAD~4</code>	Interactive rebase to squash, fixup, reword, or drop past 4 commits.
<code>git rebase --continue / git rebase --abort</code>	Advance or safely cancel ongoing rebase after conflict resolution.
<code>git push -u origin feature/orders</code>	Push branch to remote and configure upstream tracking.
<code>git tag -a v1.2.0 -m 'Release v1.2.0' && git push origin v1.2.0</code>	Create and push annotated release tag.

3.3 Advanced Git Recovery, State Manipulation & Diagnostics

<code>git stash push -m 'wip-cache' --include-untracked</code>	Stash working directory modifications including newly created files.
<code>git stash list / git stash pop / git stash drop</code>	List stashes, apply and remove latest stash, or discard stash.
<code>git cherry-pick</code>	Apply a specific commit from another branch into current branch.
<code>git cherry-pick --abort</code>	Cancel ongoing cherry-pick operation during conflict.
<code>git restore</code>	Discard uncommitted working directory changes for specified file.
<code>git restore --staged</code>	Unstage file while preserving working directory modifications.
<code>git reset --soft HEAD~1</code>	Undo last commit while preserving changes staged in index.
<code>git reset --mixed HEAD~1</code>	Undo last commit; unstage changes but preserve in working directory.
<code>git reset --hard HEAD~1</code>	Undo last commit and discard all modifications permanently.
<code>git revert -m 1</code>	Create a new inverse commit to safely undo a merged pull request.
<code>git reflog</code>	Display complete chronological log of local HEAD movements (safety net for lost commits).
<code>git reset --hard HEAD@{2}</code>	Restore branch state to where HEAD was 2 actions ago via reflog.
<code>git bisect start HEAD v1.0.0</code>	Binary search commit history to automatically pinpoint bug introduction.
<code>git clean -fdx</code>	Forcefully remove all untracked files, directories, and ignored build artifacts.

Module 4: Defensive Shell Scripting (Bash) Automation

Production shell scripts must be deterministic, defensively programmed, idempotent, and capable of structured logging and cleanup on failure.

4.1 Production Shell Scripting Standards & Constructs

<code>set -euo pipefail</code>	Exit on error (-e), exit on unset variable (-u), exit on pipeline failure (-o pipefail).
<code>IFS=\$'\n\t'</code>	Set Internal Field Separator to newline/tab to prevent accidental word-splitting on spaces.
<code>readonly SCRIPT_DIR="\$(cd "\$(dirname "\${BASH_SOURCE[0]}")" && pwd)"</code>	Determine absolute directory of executing script reliably.
<code>trap cleanup EXIT INT TERM</code>	Catch script exit, Ctrl+C (SIGINT), and SIGTERM to execute guaranteed cleanup function.
<code>\$# / \$@ / \$? / \$\$</code>	Number of args (\$#), all args (\$@), exit status of last command (\$?), current PID (\$\$).
<code>[[-f \$file]] / [[-d \$dir]] / [[-z \$str]]</code>	Check file existence (-f), directory existence (-d), empty string check (-z).

4.2 Production Backup & Cloud Archive Script (backup_manager.sh)

```
#!/usr/bin/env bash
# =====
# Script: backup_manager.sh
# Purpose: Enterprise automated file/DB backup with retention & error traps
# =====
set -euo pipefail
IFS=$'\n\t'

readonly TIMESTAMP="$(date +%Y%m%d_%H%M%S)"
readonly BACKUP_SRC="/var/www/app"
readonly BACKUP_DEST="/var/backups/app"
readonly ARCHIVE_NAME="backup_${TIMESTAMP}.tar.gz"
readonly FULL_PATH="${BACKUP_DEST}/${ARCHIVE_NAME}"
readonly RETENTION_DAYS=14
readonly LOG_FILE="/var/log/backup_manager.log"

log() {
    local level="${1}"
    local msg="${2}"
    echo "[$(date +%Y-%m-%dT%H:%M:%S%Z)] [${level}] ${msg}" | tee -a "${LOG_FILE}"
}

cleanup() {
    local exit_code=$?
    if [[ ${exit_code} -ne 0 ]]; then
        log "ERROR" "Backup execution failed with exit status code: ${exit_code}"
    fi
    exit ${exit_code}
}
trap cleanup EXIT INT TERM

main() {
    log "INFO" "Starting automated backup procedure..."
    mkdir -p "${BACKUP_DEST}"

    if [[ ! -d "${BACKUP_SRC}" ]]; then
        log "FATAL" "Source directory ${BACKUP_SRC} does not exist!"
        exit 1
    fi

    log "INFO" "Creating compressed archive: ${FULL_PATH}"
    tar -czf "${FULL_PATH}" -C "${BACKUP_SRC}" .

    log "INFO" "Archive created successfully. Size: $(du -sh "${FULL_PATH}" | cut -f1)"

    log "INFO" "Pruning archives older than ${RETENTION_DAYS} days..."
    find "${BACKUP_DEST}" -type f -name "backup_*.tar.gz" -mtime "+${RETENTION_DAYS}" -exec rm -v {} \; | while
    read -r deleted; do
        log "INFO" "Deleted old archive: ${deleted}"
    done

    log "INFO" "Backup job finished successfully."
}

main "$@"
```

Module 5: Python for DevOps, Cloud & SRE Automation

Python enables robust infrastructure orchestration, REST API integration, cloud SDK automation, and self-healing system monitors.

5.1 Production Infrastructure Auto-Healing Monitor (infra_healer.py)

```
#!/usr/bin/env python3
"""
InfraHealer: Automated multi-service health checker and self-healing engine.
Monitors systemd units, validates HTTP endpoints, and triggers alerts.
"""
import os
import sys
import time
import logging
import subprocess
import requests

logging.basicConfig(
    level=logging.INFO,
    format="[%(asctime)s] [%(levelname)s] %(message)s",
    handlers=[logging.StreamHandler(sys.stdout)]
)

SERVICES_TO_MONITOR = ["nginx", "docker", "redis-server"]
HTTP_ENDPOINTS = {
    "WebAPI": "http://localhost:8080/healthz",
    "AuthService": "http://localhost:9000/ready"
}
SLACK_WEBHOOK_URL = os.getenv("SLACK_WEBHOOK_URL")

def send_alert(message: str) -> None:
    logging.warning(f"ALERT: {message}")
    if SLACK_WEBHOOK_URL:
        try:
            requests.post(SLACK_WEBHOOK_URL, json={"text": f":warning: *DevOps Alert:* {message}"}, timeout=5)
        except Exception as err:
            logging.error(f"Failed to deliver Slack alert: {err}")

def is_systemd_active(service: str) -> bool:
    try:
        res = subprocess.run(["systemctl", "is-active", "--quiet", service], check=False)
        return res.returncode == 0
    except Exception as exc:
        logging.error(f"Error checking status for {service}: {exc}")
        return False

def restart_systemd_service(service: str) -> bool:
    logging.info(f"Attempting automated restart of service: {service}")
    try:
        res = subprocess.run(["sudo", "systemctl", "restart", service], capture_output=True, text=True, check=False)
        return res.returncode == 0
    except Exception as exc:
        logging.error(f"Restart command threw exception: {exc}")
        return False

def check_http_endpoint(name: str, url: str) -> bool:
    try:
        resp = requests.get(url, timeout=4)
        return resp.status_code == 200
    except Exception:
        return False

def run_health_cycle():
    logging.info("Starting health audit cycle...")
    for svc in SERVICES_TO_MONITOR:
        if not is_systemd_active(svc):
            send_alert(f"Service `{svc}` is DOWN. Triggering auto-recovery...")
            if restart_systemd_service(svc):
                time.sleep(2)
```

```
if is_systemd_active(svc):
    send_alert(f"white_check_mark: Service `{svc}` recovered successfully.")
else:
    send_alert(f"skull: Service `{svc}` failed to remain active after restart!")
else:
    send_alert(f"skull: Sudo restart failed for `{svc}`.")

for name, endpoint in HTTP_ENDPOINTS.items():
    if not check_http_endpoint(name, endpoint):
        send_alert(f"HTTP Probe Failed for `{name}` at `{endpoint}`.")

if __name__ == "__main__":
    run_health_cycle()
```

Module 6: Containerization with Docker & Compose Stacks

Docker provides immutable application packaging, standardized container runtimes, isolated networking, and reproducible build pipelines.

6.1 Complete Docker CLI Command Reference

<code>docker build -t app:1.0.0 -f Dockerfile .</code>	Build container image with name tag from Dockerfile in current context.
<code>docker build --no-cache --pull -t app:latest .</code>	Build image pulling fresh base layers and ignoring cached build steps.
<code>docker run -d --name web -p 8080:80 --restart=always -v data:/var/data nginx:alpine</code>	Run detached container with port mapping, auto-restart policy, and named volume.
<code>docker ps -a --format 'table {{.Names}}\t{{.Status}}\t{{.Ports}}'</code>	Formatted tabular overview of all container statuses and ports.
<code>docker logs -f --tail=100 --timestamps web</code>	Stream container standard output/error with timestamps and last 100 lines.
<code>docker exec -it web /bin/sh</code>	Execute interactive shell session inside active container.
<code>docker exec -u 0 -it web whoami</code>	Execute command inside container as root user (UID 0).
<code>docker stop web && docker rm web</code>	Gracefully stop container (SIGTERM->SIGKILL) and remove container object.
<code>docker kill --signal=HUP web</code>	Send specific POSIX signal (SIGHUP) to main process in container.
<code>docker inspect --format '{{range .NetworkSettings.Networks}}{{.IPAddress}}{{end}}' web</code>	Extract internal IP address of container using Go template.
<code>docker stats --no-stream</code>	Single-shot snapshot of CPU, memory, network I/O, and block I/O per container.
<code>docker top web</code>	Display host PIDs and processes running inside container namespace.
<code>docker cp web:/etc/nginx/nginx.conf ./nginx.conf</code>	Copy file out of running container to host filesystem.
<code>docker network create --driver bridge app_net</code>	Create dedicated isolated bridge network for inter-container communication.
<code>docker network connect app_net db_container</code>	Dynamically connect running container to network.
<code>docker volume create pg_data</code>	Create managed persistent storage volume decoupled from container lifecycle.
<code>docker system prune -af --volumes</code>	Deep cleanup purging all stopped containers, unused networks, images, and volumes.

6.2 Multi-Stage Production Dockerfile (Node.js & Nginx)

```
# =====  
# Stage 1: Build & Compilation (Node.js 20 Alpine)  
# =====  
FROM node:20-alpine AS builder  
WORKDIR /usr/src/app  
  
# Leverage layer cache for dependencies  
COPY package*.json ./  
RUN npm ci --only=production  
  
COPY . .  
RUN npm run build  
  
# =====  
# Stage 2: Hardened Runtime (Nginx 1.27 Slim)  
# =====  
FROM nginx:1.27-alpine-slim  
LABEL maintainer="devops@company.com"  
  
# Purge default assets  
RUN rm -rf /usr/share/nginx/html/*  
  
# Copy build artifacts and configuration  
COPY --from=builder /usr/src/app/dist /usr/share/nginx/html  
COPY nginx.conf /etc/nginx/conf.d/default.conf  
  
# Security: Run as non-root user  
USER nginx  
EXPOSE 8080  
  
HEALTHCHECK --interval=30s --timeout=3s --retries=3 \  
CMD wget --quiet --tries=1 --spider http://localhost:8080/health || exit 1  
  
CMD ["nginx", "-g", "daemon off;"]
```

6.3 Production 3-Tier Docker Compose (docker-compose.yml)

```
services:
  app:
    build:
      context: .
      dockerfile: Dockerfile
    ports:
      - "8080:8080"
    environment:
      - DATABASE_URL=postgres://db_user:secret123@postgres:5432/app_db
      - REDIS_URL=redis://redis:6379/0
    depends_on:
      postgres:
        condition: service_healthy
      redis:
        condition: service_started
    networks:
      - backend_net

  postgres:
    image: postgres:16-alpine
    environment:
      POSTGRES_USER: db_user
      POSTGRES_PASSWORD: secret123
      POSTGRES_DB: app_db
    volumes:
      - pg_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U db_user -d app_db"]
      interval: 10s
      timeout: 5s
      retries: 5
    networks:
      - backend_net

  redis:
    image: redis:7-alpine
    command: ["redis-server", "--appendonly", "yes"]
    volumes:
      - redis_data:/data
    networks:
      - backend_net

volumes:
  pg_data:
  redis_data:

networks:
  backend_net:
    driver: bridge
```

Module 7: Kubernetes Orchestration & Helm Packaging

Kubernetes is the enterprise standard for container orchestration, declarative configuration, self-healing, rolling deployments, and automated scaling.

7.1 Complete kubectl Command Suite

<code>kubectl cluster-info</code>	Display control plane master and CoreDNS endpoints.
<code>kubectl config get-contexts / use-context</code>	List available kubeconfig contexts and switch active cluster context.
<code>kubectl get nodes -o wide</code>	List all cluster worker nodes with internal/external IPs, OS, and Kubelet versions.
<code>kubectl top nodes && kubectl top pods -A</code>	View live CPU and memory resource consumption per node and per pod.
<code>kubectl get pods -A --field-selector=status.phase!=Running</code>	Filter and find all failing/crashing pods across all namespaces.
<code>kubectl apply -f manifest.yaml</code>	Declaratively create or update resources from YAML manifest.
<code>kubectl apply -k ./overlays/production</code>	Apply Kustomize overlay for environment-specific manifests.
<code>kubectl describe pod -n prod</code>	Inspect pod lifecycle events, image pull errors, and volume mount details.
<code>kubectl logs -f deployment/api-server -c app -n prod</code>	Follow logs of specific container within a deployment.
<code>kubectl logs --previous pod-crash-xxx -n prod</code>	Extract logs from previous terminated container instance after crash.
<code>kubectl exec -it -n prod -- /bin/sh</code>	Attach interactive shell session inside pod container.
<code>kubectl scale deployment/api-server --replicas=5 -n prod</code>	Imperatively scale replica count of deployment.
<code>kubectl rollout restart deployment/api-server -n prod</code>	Trigger rolling restart of pods with zero downtime.
<code>kubectl rollout status deployment/api-server -n prod</code>	Check status of rolling update deployment.
<code>kubectl rollout history deployment/api-server -n prod</code>	View revision history of deployment rollouts.
<code>kubectl rollout undo deployment/api-server --to-revision=2 -n prod</code>	Rollback deployment to specified previous revision.
<code>kubectl port-forward svc/api-service 8080:80 -n prod</code>	Forward local port 8080 directly to in-cluster service port 80.
<code>kubectl get events --sort-by='.metadata.creationTimestamp' -A</code>	List all cluster events chronologically to diagnose issues.
<code>kubectl drain --ignore-daemonsets --delete-emptydir-data</code>	Safely evict pods from node before maintenance.
<code>kubectl cordon / uncordon</code>	Mark node unschedulable or return to active pool.

7.2 Production Kubernetes Deployment, Service & Probes

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: api-service
  namespace: production
  labels:
    app.kubernetes.io/name: api-service
spec:
  replicas: 3
  strategy:
    type: RollingUpdate
  rollingUpdate:
    maxSurge: 1
    maxUnavailable: 0
  selector:
    matchLabels:
      app: api-service
  template:
    metadata:
      labels:
        app: api-service
    spec:
      containers:
        - name: api
          image: ghcr.io/company/api:v2.1.0
          imagePullPolicy: IfNotPresent
          ports:
            - containerPort: 8080
          resources:
            requests:
              cpu: "100m"
              memory: "128Mi"
            limits:
              cpu: "500m"
              memory: "512Mi"
          readinessProbe:
            httpGet:
              path: /healthz
              port: 8080
            initialDelaySeconds: 5
            periodSeconds: 10
          livenessProbe:
            httpGet:
              path: /healthz
              port: 8080
            initialDelaySeconds: 15
            periodSeconds: 20
            ---
apiVersion: v1
kind: Service
metadata:
  name: api-service
  namespace: production
spec:
  type: ClusterIP
  selector:
    app: api-service
  ports:
    - port: 80
      targetPort: 8080
```

7.3 Helm 3 Package Management Reference

<code>helm repo add bitnami https://charts.bitnami.com/bitnami</code>	Add remote Helm chart repository index.
<code>helm repo update</code>	Fetch latest version metadata from all configured Helm repositories.
<code>helm search repo redis</code>	Search repositories for available charts and version numbers.
<code>helm install my-redis bitnami/redis -f values-prod.yaml -n prod --create-namespace</code>	Deploy release with custom values override file in target namespace.
<code>helm upgrade --install my-redis bitnami/redis -f values-prod.yaml -n prod</code>	Idempotent upgrade or install of release.
<code>helm history my-redis -n prod</code>	List all past release revisions, timestamps, and deployment statuses.
<code>helm rollback my-redis 2 -n prod</code>	Instantly rollback release to revision 2.
<code>helm list -A</code>	List all deployed releases across all Kubernetes namespaces.
<code>helm template my-release ./mychart -f values.yaml</code>	Render chart templates locally into raw YAML without applying.
<code>helm lint ./mychart</code>	Run linter examining chart syntax and best practices.

Module 8: Infrastructure as Code (IaC) with Terraform

Terraform and OpenTofu provide declarative cloud infrastructure provisioning, state locking, module reusability, and automated drift management.

8.1 Terraform CLI Workflow & State Operations

<code>terraform init -upgrade</code>	Initialize backend, download provider plugins, and upgrade module versions.
<code>terraform fmt -recursive</code>	Automatically format all HCL configuration files to standard styling.
<code>terraform validate</code>	Validate syntactic correctness and internal variable references.
<code>terraform plan -out=tfplan.binary</code>	Generate speculative execution plan and save immutable binary artifact.
<code>terraform apply tfplan.binary</code>	Apply exact changes recorded in the saved plan binary file.
<code>terraform destroy</code> <code>-target=aws_instance.bastion</code>	Targeted destruction of a single resource without affecting stack.
<code>terraform state list</code>	List all resources currently tracked in the remote state file.
<code>terraform state show aws_vpc.main</code>	Inspect full attributes and metadata of tracked state resource.
<code>terraform state mv aws_instance.old</code> <code>aws_instance.new</code>	Rename resource address in state without destroying infrastructure.
<code>terraform import aws_vpc.main</code> <code>vpc-0a1b2c3d4e5f6g7h8</code>	Import existing cloud resource into Terraform state.
<code>terraform workspace new prod &&</code> <code>terraform workspace select prod</code>	Create and switch to isolated production workspace state.
<code>terraform output -json</code>	Export configuration outputs in structured JSON format.

8.2 Modular Production AWS VPC Terraform Code (main.tf)

```
terraform {
  required_version = ">= 1.8.0"
  required_providers {
    aws = {
      source = "hashicorp/aws"
      version = "~> 5.50"
    }
  }
}

backend "s3" {
  bucket = "company-tf-state-prod"
  key = "vpc/terraform.tfstate"
  region = "us-east-1"
  dynamodb_table = "terraform-state-lock"
  encrypt = true
}

provider "aws" {
  region = var.aws_region
  default_tags {
    tags = {
      Environment = var.environment
      ManagedBy = "Terraform"
    }
  }
}

resource "aws_vpc" "main" {
  cidr_block = var.vpc_cidr
  enable_dns_hostnames = true
  enable_dns_support = true
  tags = { Name = "${var.environment}-vpc" }
}

resource "aws_subnet" "public" {
  count = length(var.public_subnet_cidrs)
  vpc_id = aws_vpc.main.id
  cidr_block = var.public_subnet_cidrs[count.index]
  availability_zone = data.aws_availability_zones.available.names[count.index]
  map_public_ip_on_launch = true
  tags = { Name = "${var.environment}-public-${count.index + 1}" }
}

resource "aws_internet_gateway" "gw" {
  vpc_id = aws_vpc.main.id
  tags = { Name = "${var.environment}-igw" }
}
```

Module 9: Configuration Management with Ansible

Ansible provides agentless, idempotent configuration management, server orchestration, and package deployment over SSH.

9.1 Ansible Ad-Hoc Commands & Inventory Management

<code>ansible all -i inventory.ini -m ping</code>	Test SSH reachability and Python interpreter on all inventory hosts.
<code>ansible webserver -i inventory.ini -m setup -a 'filter=ansible_default_ipv4'</code>	Gather system facts (IP address) from target host group.
<code>ansible webserver -i inventory.ini -m apt -a 'name=nginx state=latest update_cache=yes' --become</code>	Install latest nginx package across group using sudo.
<code>ansible all -i inventory.ini -m command -a 'uptime'</code>	Execute arbitrary shell command across all hosts.
<code>ansible-playbook -i inventory.ini site.yml --syntax-check</code>	Validate YAML syntax of playbook and referenced roles.
<code>ansible-playbook -i inventory.ini site.yml --check --diff</code>	Dry-run execution displaying exact line-by-line configuration diffs.
<code>ansible-playbook -i inventory.ini site.yml -l webserver -t 'nginx'</code>	Execute only tasks tagged with 'nginx' on 'webserver' host group.
<code>ansible-vault encrypt_string 'db_password' --name 'db_pass'</code>	Encrypt inline sensitive strings using AES256 vault password.
<code>ansible-vault edit secrets.yml --vault-password-file .vault_pass</code>	Edit encrypted YAML secrets file in-place.

9.2 Production Web Server Hardening Playbook (site.yml)

```
---
- name: Provision & Harden Production Web Servers
  hosts: webservers
  become: yes
  vars:
    app_user: deployer

  tasks:
    - name: Update apt package cache and upgrade system
      apt:
        update_cache: yes
        upgrade: dist
        cache_valid_time: 3600

    - name: Install baseline system packages
      apt:
        name: [nginx, ufw, curl, git, htop, fail2ban]
        state: present

    - name: Ensure deployment user exists
      user:
        name: "{{ app_user }}"
        shell: /bin/bash
        groups: sudo
        append: yes

    - name: Deploy custom Nginx virtualhost template
      template:
        src: templates/nginx.conf.j2
        dest: /etc/nginx/sites-available/default
        mode: '0644'
        notify: Restart Nginx

    - name: Configure UFW firewall rules
      ufw:
        rule: allow
        port: "{{ item }}"
        proto: tcp
        loop: [22, 80, 443]

  handlers:
    - name: Restart Nginx
      service:
        name: nginx
        state: restarted
        enabled: yes
```

Module 10: Enterprise CI/CD Pipelines (GHA & Jenkins)

Automate continuous integration, vulnerability scanning, container compilation, and GitOps deployments with GitHub Actions and Jenkins pipelines.

10.1 Enterprise GitHub Actions Pipeline (.github/workflows/deploy.yml)

```
name: Enterprise CI/CD Pipeline

on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]
  env:
    REGISTRY: ghcr.io
    IMAGE_NAME: ${ github.repository }

jobs:
  test:
    name: Lint & Unit Tests
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: '20'
          cache: 'npm'
      - run: |
          npm ci
          npm run lint
          npm test -- --coverage

  build-and-push:
    name: Docker Build, Scan & Push
    needs: test
    if: github.ref == 'refs/heads/main'
    runs-on: ubuntu-latest
    permissions:
      contents: read
      packages: write
    steps:
      - uses: actions/checkout@v4
      - uses: docker/login-action@v3
        with:
          registry: ${ env.REGISTRY }
          username: ${ github.actor }
          password: ${ secrets.GITHUB_TOKEN }
      - uses: docker/build-push-action@v5
        with:
          context: .
          push: true
          tags: ${ env.REGISTRY }/${ env.IMAGE_NAME }:${ github.sha },${ env.REGISTRY }/${ env.IMAGE_NAME }:latest

  deploy:
    name: Deploy to Kubernetes Cluster
    needs: build-and-push
    runs-on: ubuntu-latest
    steps:
      - uses: azure/k8s-set-context@v3
        with:
          kubeconfig: ${ secrets.KUBECONFIG_PROD }
      - run: |
          kubectl set image deployment/web-app web-app=${ env.REGISTRY }/${ env.IMAGE_NAME }:${ github.sha } -n production
          kubectl rollout status deployment/web-app -n production --timeout=180s
```

10.2 Declarative Jenkins Pipeline (Jenkinsfile)

```
pipeline {
  agent any
  environment {
    DOCKER_IMAGE = "registry.company.com/app:${BUILD_NUMBER}"
  }
  stages {
    stage('Checkout & Lint') {
      steps {
        checkout scm
        sh 'npm ci && npm run lint'
      }
    }
    stage('Test') {
      steps {
        sh 'npm test'
      }
    }
    stage('Build Docker Image') {
      steps {
        sh "docker build -t ${DOCKER_IMAGE} ."
      }
    }
    stage('Deploy to Production') {
      steps {
        withKubeConfig([credentialsId: 'k8s-prod-creds']) {
          sh "kubectl set image deployment/api-server api=${DOCKER_IMAGE} -n production"
          sh "kubectl rollout status deployment/api-server -n production"
        }
      }
    }
  }
  post {
    failure {
      echo "Pipeline failed! Dispatching alerts..."
    }
  }
}
```

Module 11: Observability, PromQL & Production Debugging

Observability is the capability to infer internal application and infrastructure health through real-time metrics, golden signals, and kernel diagnostics.

11.1 Prometheus Configuration (prometheus.yml)

```
global:
  scrape_interval: 15s
  evaluation_interval: 15s

scrape_configs:
  - job_name: 'node_exporter'
    static_configs:
      - targets: ['10.0.0.10:9100', '10.0.0.11:9100']

  - job_name: 'kubernetes-pods'
    kubernetes_sd_configs:
      - role: pod
    relabel_configs:
      - source_labels: [__meta_kubernetes_pod_annotation_prometheus_io_scrape]
        action: keep
        regex: true
      - source_labels: [__meta_kubernetes_pod_annotation_prometheus_io_port]
        action: replace
        target_label: __address__
        regex: (.+)
        replacement: ${1}
```

11.2 Essential PromQL Metrics Queries (Golden Signals)

<code>100 - (avg by (instance) (rate(node_cpu_seconds_total{mode='idle'}[5m])) * 100)</code>	Real-time CPU utilization percentage per server instance.
<code>(node_memory_MemTotal_bytes - node_memory_MemAvailable_bytes) / node_memory_MemTotal_bytes * 100</code>	Current RAM memory utilization percentage.
<code>sum(rate(http_requests_total{status=~'5..'}[5m])) / sum(rate(http_requests_total[5m])) * 100</code>	HTTP 5xx Server Error Rate percentage (Golden Signal: Errors).
<code>histogram_quantile(0.95, sum(rate(http_request_duration_seconds_bucket[5m])) by (le))</code>	95th percentile (p95) API response latency in seconds.
<code>sum(container_memory_working_set_bytes{namespace='production'}) by (pod)</code>	Resident memory consumed per Kubernetes pod container.
<code>rate(container_network_receive_bytes_total[5m])</code>	Network receive throughput in bytes per second per container.

11.3 Advanced Linux & Production Troubleshooting Toolkit

<code>lsof -i :8080</code>	Identify process name, user, and PID holding port 8080 open.
<code>lsof -p</code>	List every file descriptor, network socket, and library opened by process.
<code>strace -f -p -e trace=network,file -s 256</code>	Attach to running process to trace syscalls (file I/O, socket connections).
<code>iotop -oP</code>	Real-time disk I/O monitoring showing only processes actively performing reads/writes.
<code>nethogs eth0</code>	Display real-time network bandwidth consumption grouped by process PID.
<code>sysctl -p /etc/sysctl.conf</code>	Apply kernel runtime parameter changes (e.g. max connections, IP forwarding).

Module 12: Enterprise Capstone & 12-Week Roadmap

End-to-end integration uniting version control, CI/CD, GitOps, Kubernetes, Infrastructure as Code, and SRE Observability.

12.1 Enterprise GitOps Architecture Blueprint



12.2 Production Readiness Review (PRR) Checklist

Security & IAM	Enforce Least Privilege IAM roles; zero hardcoded secrets; use Vault/SealedSecrets; encrypt disks & transit.
High Availability	Multi-AZ deployment; minimum 3 replicas per service; PodDisruptionBudgets configured; auto-scaling active.
Observability	Liveness & readiness probes configured; Prometheus metrics exposed; centralized structured JSON logs enabled.
Disaster Recovery	Automated daily snapshot backups tested for point-in-time recovery; multi-region backup replication.
Capacity Planning	CPU/Memory resource requests and limits set on every container; HorizontalPodAutoscaler (HPA) active.

12.3 12-Week Zero-to-Hero Mastery Action Plan

Weeks 1 - 2	Linux SysAdmin, Bash Scripting, File Permissions, Text Processing, Socket Inspection & Networking.
Weeks 3 - 4	Advanced Git Workflows, Interactive Rebasing, Reflog Diagnostics & Python SRE Automation Scripts.
Weeks 5 - 6	Docker Containerization, Multi-Stage Builds, Layer Caching Optimization & Docker Compose Stacks.
Weeks 7 - 8	Kubernetes Core Architecture, Deployments, Probes, Services, Ingress, Helm & Cluster Debugging.
Weeks 9 - 10	Infrastructure as Code with Terraform AWS Modules & Configuration Management with Ansible.
Weeks 11 - 12	Enterprise CI/CD Pipelines (GitHub Actions/Jenkins), Prometheus/Grafana Monitoring & Capstone.